

```
In [54]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

air_quality = pd.read_csv("city_day.csv")
crop_production = pd.read_csv("crop_production.csv")
```

```
In [55]: air_quality.head()
```

```
Out[55]:
```

	City	Date	PM2.5	PM10	NO	NO2	NOx	NH3	CO	SO2	O3	Benz
0	Ahmedabad	2015-01-01	NaN	NaN	0.92	18.22	17.15	NaN	0.92	27.64	133.36	
1	Ahmedabad	2015-01-02	NaN	NaN	0.97	15.69	16.46	NaN	0.97	24.55	34.06	
2	Ahmedabad	2015-01-03	NaN	NaN	17.40	19.30	29.70	NaN	17.40	29.07	30.70	
3	Ahmedabad	2015-01-04	NaN	NaN	1.70	18.48	17.97	NaN	1.70	18.59	36.08	
4	Ahmedabad	2015-01-05	NaN	NaN	22.10	21.42	37.76	NaN	22.10	39.33	39.31	

```
In [56]: air_quality.shape
```

```
Out[56]: (29531, 16)
```

```
In [57]: air_quality.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 29531 entries, 0 to 29530
Data columns (total 16 columns):
#   Column      Non-Null Count  Dtype
---  -
0   City         29531 non-null  object
1   Date         29531 non-null  object
2   PM2.5        24933 non-null  float64
3   PM10         18391 non-null  float64
4   NO           25949 non-null  float64
5   NO2          25946 non-null  float64
6   NOx          25346 non-null  float64
7   NH3          19203 non-null  float64
8   CO           27472 non-null  float64
9   SO2          25677 non-null  float64
10  O3           25509 non-null  float64
11  Benzene      23908 non-null  float64
12  Toluene      21490 non-null  float64
13  Xylene       11422 non-null  float64
14  AQI          24850 non-null  float64
15  AQI_Bucket   24850 non-null  object
dtypes: float64(13), object(3)
memory usage: 3.6+ MB

```

In [58]: `air_quality.describe()`

Out[58]:

	PM2.5	PM10	NO	NO2	NOx	NH3
count	24933.000000	18391.000000	25949.000000	25946.000000	25346.000000	19203.000000
mean	67.450578	118.127103	17.574730	28.560659	32.309123	23.483476
std	64.661449	90.605110	22.785846	24.474746	31.646011	25.684275
min	0.040000	0.010000	0.020000	0.010000	0.000000	0.010000
25%	28.820000	56.255000	5.630000	11.750000	12.820000	8.580000
50%	48.570000	95.680000	9.890000	21.690000	23.520000	15.850000
75%	80.590000	149.745000	19.950000	37.620000	40.127500	30.020000
max	949.990000	1000.000000	390.680000	362.210000	467.630000	352.890000

In [59]: `air_quality.isnull().sum()`

```
Out[59]: City          0
Date          0
PM2.5        4598
PM10         11140
NO           3582
NO2          3585
NOx          4185
NH3          10328
CO           2059
SO2          3854
O3           4022
Benzene      5623
Toluene      8041
Xylene       18109
AQI          4681
AQI_Bucket   4681
dtype: int64
```

```
In [60]: crop_production.head()
```

```
Out[60]:
```

	State_Name	District_Name	Crop_Year	Season	Crop	Area	Production
0	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	Arecanut	1254.0	2000.0
1	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	Other Kharif pulses	2.0	1.0
2	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	Rice	102.0	321.0
3	Andaman and Nicobar Islands	NICOBARS	2000	Whole Year	Banana	176.0	641.0
4	Andaman and Nicobar Islands	NICOBARS	2000	Whole Year	Cashewnut	720.0	165.0

```
In [61]: crop_production.shape
```

```
Out[61]: (246091, 7)
```

```
In [62]: crop_production.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 246091 entries, 0 to 246090
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   State_Name      246091 non-null object
1   District_Name   246091 non-null object
2   Crop_Year       246091 non-null int64
3   Season          246091 non-null object
4   Crop            246091 non-null object
5   Area            246091 non-null float64
6   Production      242361 non-null float64
dtypes: float64(2), int64(1), object(4)
memory usage: 13.1+ MB

```

In [63]: `crop_production.describe()`

Out[63]:

	Crop_Year	Area	Production
count	246091.000000	2.460910e+05	2.423610e+05
mean	2005.643018	1.200282e+04	5.825034e+05
std	4.952164	5.052340e+04	1.706581e+07
min	1997.000000	4.000000e-02	0.000000e+00
25%	2002.000000	8.000000e+01	8.800000e+01
50%	2006.000000	5.820000e+02	7.290000e+02
75%	2010.000000	4.392000e+03	7.023000e+03
max	2015.000000	8.580100e+06	1.250800e+09

In [64]: `crop_production.isnull().sum()`

Out[64]:

State_Name	0
District_Name	0
Crop_Year	0
Season	0
Crop	0
Area	0
Production	3730

dtype: int64

Observation:

The AQI dataset contains several missing pollutant readings, especially PM10 and AQI values. Removing all rows would cause major data loss.

The crop dataset is mostly complete but Production has missing values that must be treated before analysis.

Outliers may also exist because environmental and agricultural measurements can have extreme values.

TASK 2: Missing Value Handling

AQI Dataset:

- Numerical pollutant columns contain continuous measurements.
- Median imputation is used because pollution values may contain extreme readings and median is less affected by outliers.

AQI_Bucket:

- It is categorical data, therefore mode value is used.

Crop Dataset:

- Production is numerical agricultural data.
- Median replacement is selected because crop production varies greatly between regions.

Rows are not removed because missing data percentage is significant.

```
In [65]: #AQI Treatment:
numeric_cols = air_quality.select_dtypes(include=np.number).columns

for col in numeric_cols:
    air_quality[col] = air_quality[col].fillna(air_quality[col].median())
```

```
In [66]: #AQI_Bucket:
air_quality["AQI_Bucket"] = air_quality["AQI_Bucket"].fillna(
    air_quality["AQI_Bucket"].mode()[0]
)
```

```
In [67]: #Crop Production Treatment:
crop_production["Production"] = crop_production["Production"].fillna(
    crop_production["Production"].median()
)
```

```
In [68]: air_quality.isnull().sum()
```

```
Out[68]: City          0
         Date          0
         PM2.5        0
         PM10         0
         NO           0
         NO2          0
         NOx          0
         NH3          0
         CO           0
         SO2          0
         O3           0
         Benzene       0
         Toluene       0
         Xylene        0
         AQI           0
         AQI_Bucket    0
         dtype: int64
```

```
In [69]: crop_production.isnull().sum()
```

```
Out[69]: State_Name      0
         District_Name   0
         Crop_Year       0
         Season          0
         Crop            0
         Area            0
         Production      0
         dtype: int64
```

TASK 3: DATA CLEANING AND STANDARDIZATION

To prepare for merging, we must ensure that the geographical names in both datasets are consistent. This includes fixing case sensitivity, removing white spaces, and mapping cities in the AQI dataset to their respective states found in the crop dataset.

```
In [70]: for df in [air_quality, crop_production]:
         for col in df.select_dtypes(include=['object']).columns:
             df[col] = df[col].str.strip().str.title()

state_map = {
    'Ahmedabad': 'Gujarat', 'Aizawl': 'Mizoram', 'Amaravati': 'Andhra Pradesh',
```

```

    'Amritsar': 'Punjab', 'Bengaluru': 'Karnataka', 'Bhopal': 'Madhya Pradesh',
    'Brajrajnagar': 'Odisha', 'Chandigarh': 'Chandigarh', 'Chennai': 'Tamil Nadu',
    'Coimbatore': 'Tamil Nadu', 'Delhi': 'Delhi', 'Ernakulam': 'Kerala',
    'Gurugram': 'Haryana', 'Guwahati': 'Assam', 'Hyderabad': 'Telangana',
    'Jaipur': 'Rajasthan', 'Jorapokhar': 'Jharkhand', 'Kochi': 'Kerala',
    'Kolkata': 'West Bengal', 'Lucknow': 'Uttar Pradesh', 'Mumbai': 'Maharashtra',
    'Patna': 'Bihar', 'Shillong': 'Meghalaya', 'Talcher': 'Odisha',
    'Thiruvananthapuram': 'Kerala', 'Visakhapatnam': 'Andhra Pradesh'
}

air_quality['State_Name'] = air_quality['City'].map(state_map)

print(f"Records in Crop Data before: {len(crop_production)}")
crop_production.drop_duplicates(inplace=True)
print(f"Records in Crop Data after: {len(crop_production)}")

print(f"Records in AQI Data before: {len(air_quality)}")
air_quality.drop_duplicates(inplace=True)
print(f"Records in AQI Data after: {len(air_quality)}")

```

```

Records in Crop Data before: 246091
Records in Crop Data after: 246091
Records in AQI Data before: 29531
Records in AQI Data after: 29531

```

Data Cleaning Summary Standardization: All categorical columns were converted to Title Case and stripped of whitespace to prevent 'Delhi' and ' delhi' from being treated as different entities. Mapping: A new State_Name column was added to the Air Quality dataset. Deduplication: Duplicate records were removed. These steps are critical: without geographical alignment (Mapping) and naming consistency.

```

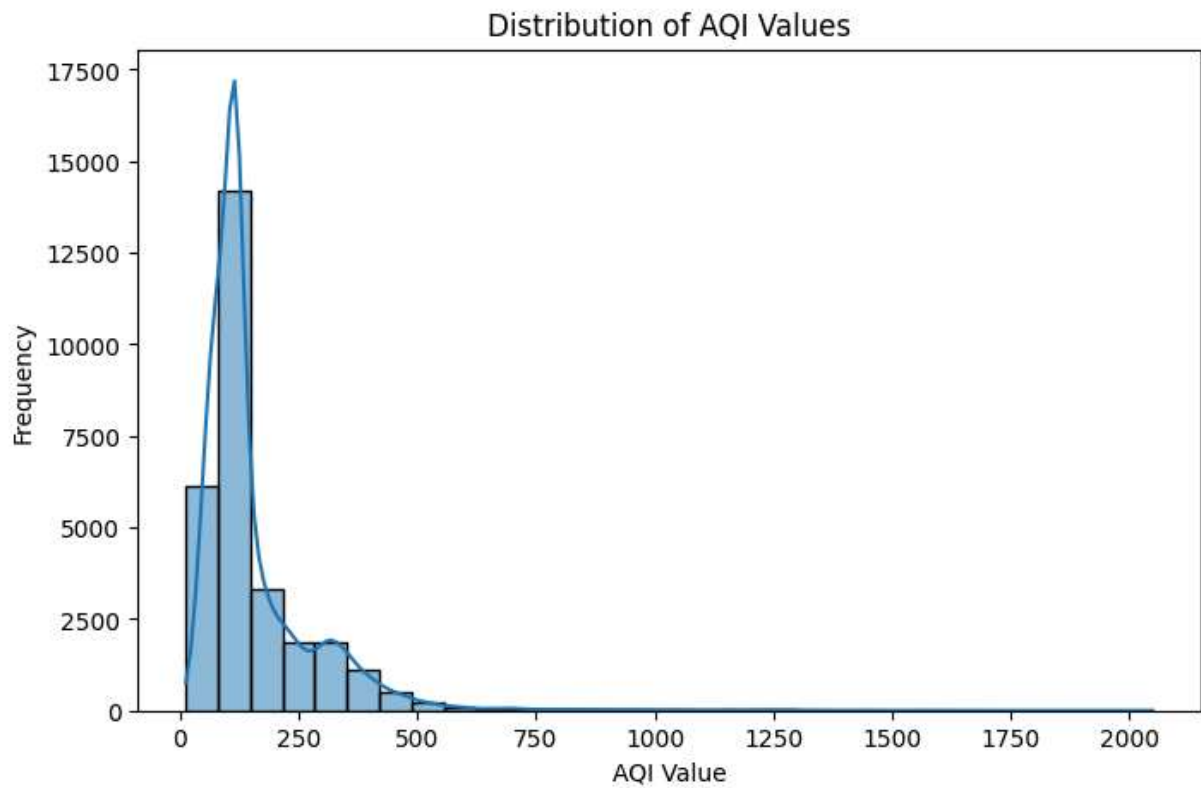
In [73]: #DISTRIBUTION ANALYSIS
plt.figure(figsize=(8,5))

sns.histplot(
    air_quality["AQI"],
    bins=30,
    kde=True
)

plt.xlabel("AQI Value")
plt.ylabel("Frequency")
plt.title("Distribution of AQI Values")

plt.show()

```

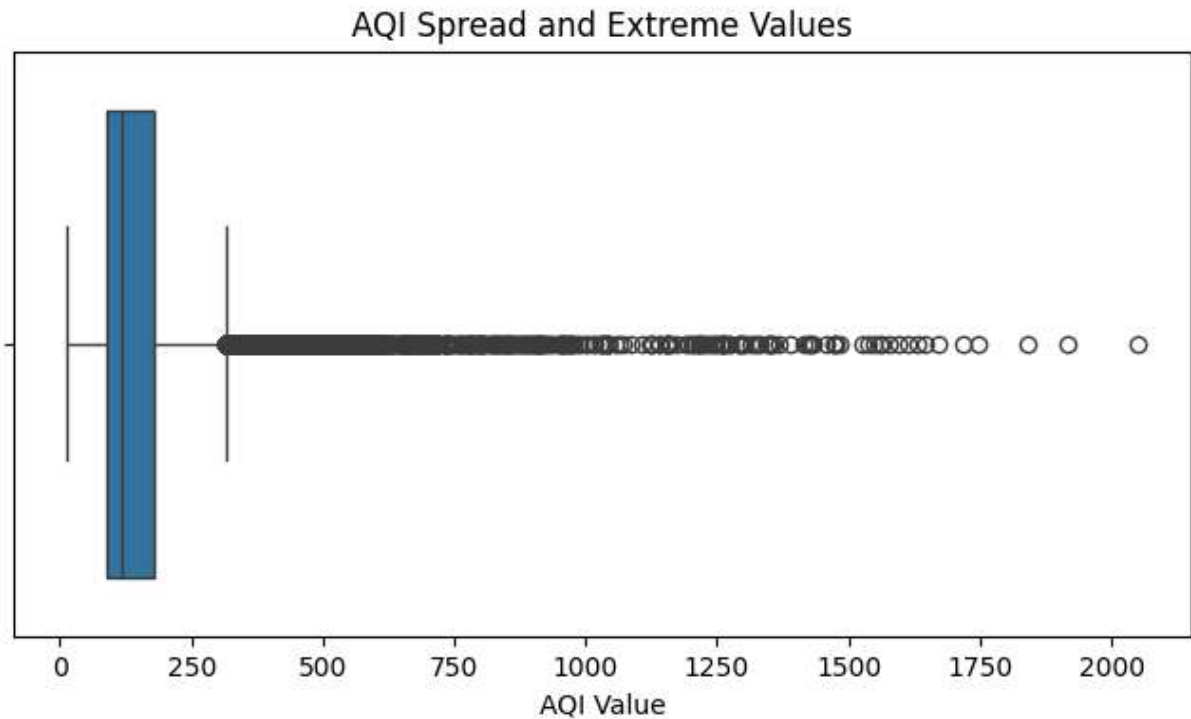


```
In [74]: plt.figure(figsize=(8,4))

sns.boxplot(
    x=air_quality["AQI"]
)

plt.xlabel("AQI Value")
plt.title("AQI Spread and Extreme Values")

plt.show()
```

Visualization Justification:

Histogram is selected because it shows where most AQI readings are concentrated.

Boxplot is selected because it clearly identifies extreme pollution values.

Observations:

1. Most AQI values are concentrated in the lower and moderate range.
2. Some very high AQI readings exist which increase the average AQI.

In [75]: *# Task 5: Outlier Detection and Treatment*

```
# AQI before treatment

plt.figure(figsize=(8,4))

sns.boxplot(
    x=air_quality["AQI"]
)

plt.title("AQI Before Outlier Treatment")
plt.xlabel("AQI Value")

plt.show()

# Calculate IQR

Q1 = air_quality["AQI"].quantile(0.25)
```

```
Q3 = air_quality["AQI"].quantile(0.75)

IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR

upper_limit = Q3 + 1.5 * IQR

# Identify extreme values

outliers = air_quality[
    (air_quality["AQI"] < lower_limit) |
    (air_quality["AQI"] > upper_limit)
]

print("Number of extreme AQI values:",
      outliers.shape[0])

# Treat outliers using capping method

air_quality["AQI_cleaned"] = np.where(
    air_quality["AQI"] > upper_limit,
    upper_limit,
    air_quality["AQI"]
)

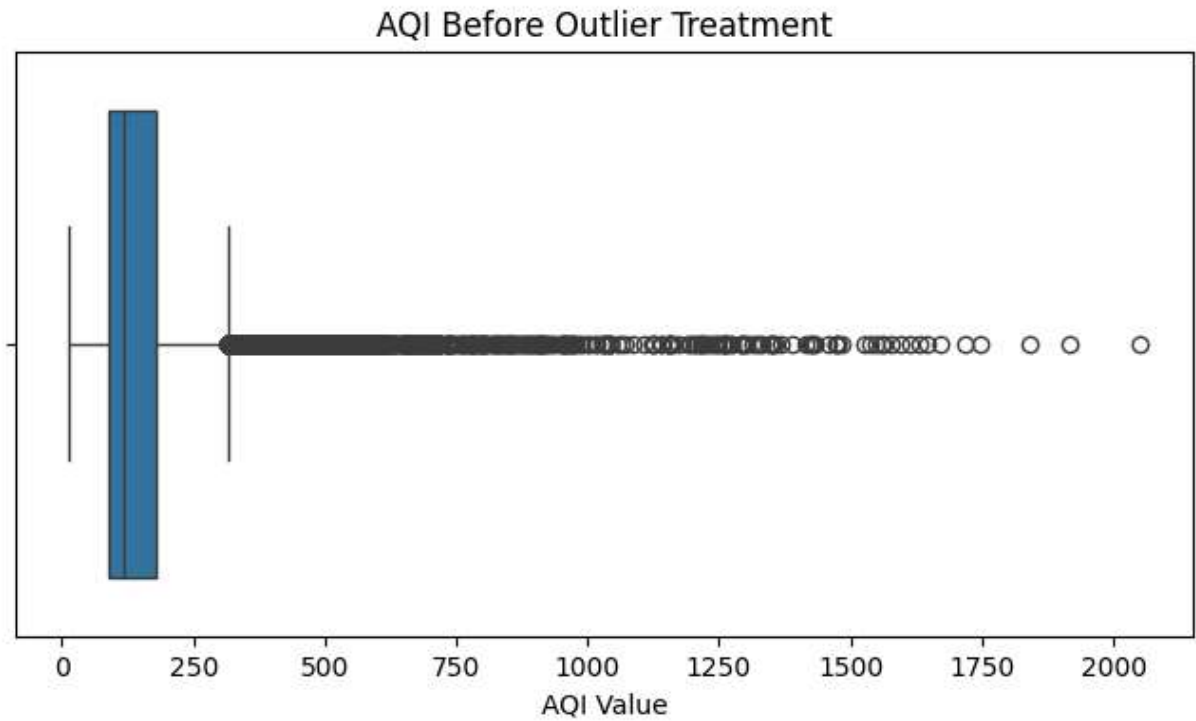
# After treatment visualization

plt.figure(figsize=(8,4))

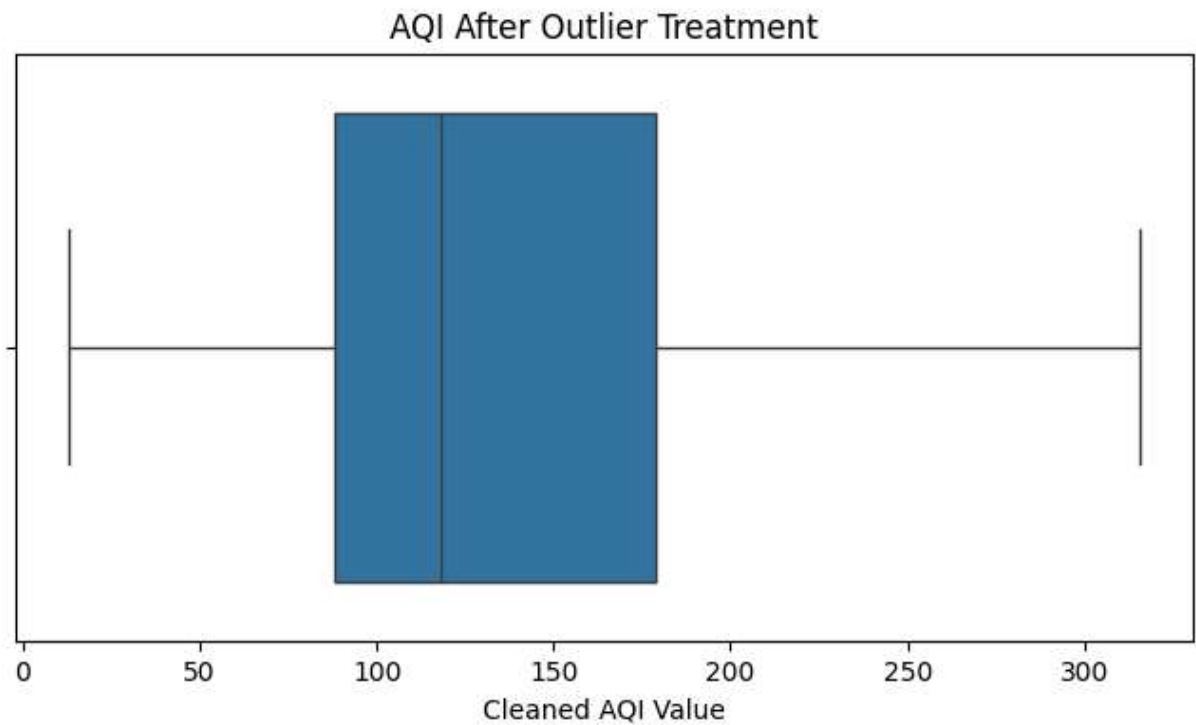
sns.boxplot(
    x=air_quality["AQI_cleaned"]
)

plt.title("AQI After Outlier Treatment")
plt.xlabel("Cleaned AQI Value")

plt.show()
```



Number of extreme AQI values: 3192



Extreme values were detected using the IQR method.

Instead of deleting records, capping was applied because very polluted days may be real environmental events.

Capping reduces the influence of extreme values while preserving the dataset size for future machine learning.